

Machine Learning for Business

GTSC2143

Lecture 3 Data Visualization

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

Why Visualization?

- Record information
- Analyze data to support reasoning
- Confirm hypotheses
- Communicate ideas to others

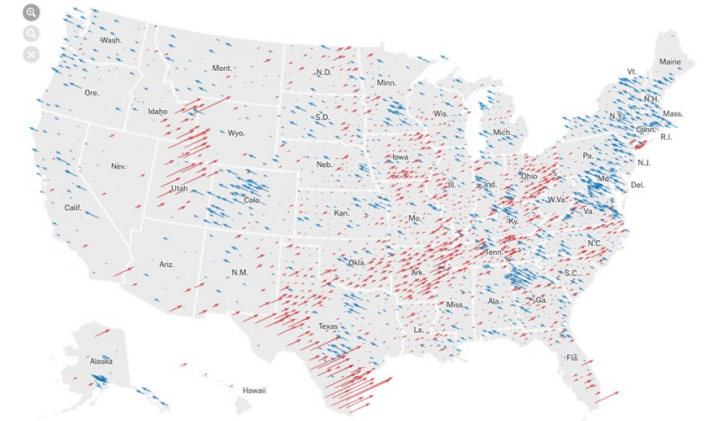
To record information



To point out interesting things

To communicate information

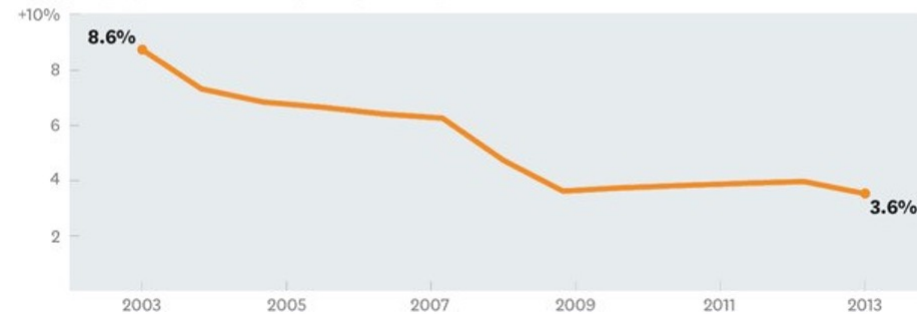
To analyze data



2020 US Elections (NYTimes)

Annual Growth is Declining

ANNUAL GROWTH IN HEALTH CARE SPENDING



MTHI~~V~~LWYADCEQGHKILKMTWYN
ARDCAIREQGHL~~V~~KMFPSTWYARN
GFPS~~V~~CEILQGKMFPSPNDRCEQDIFP
SGHLMFHKM~~V~~PSTWYACEQTWRN

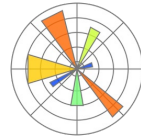
Useful Python Libraries



NumPy



pandas



matplotlib

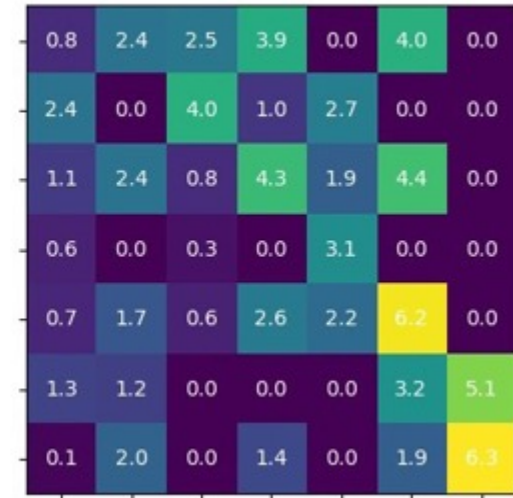
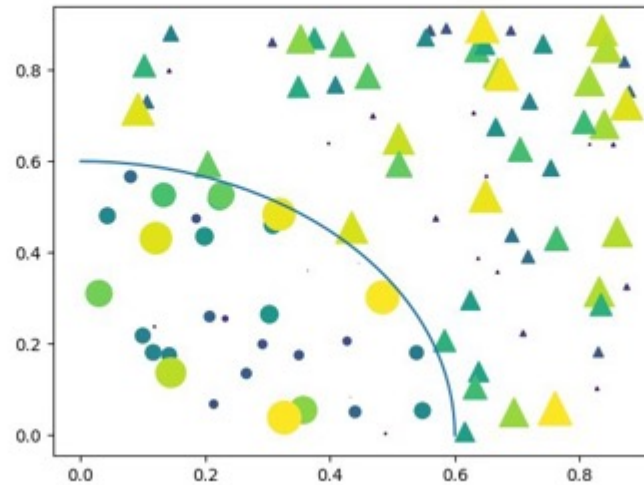
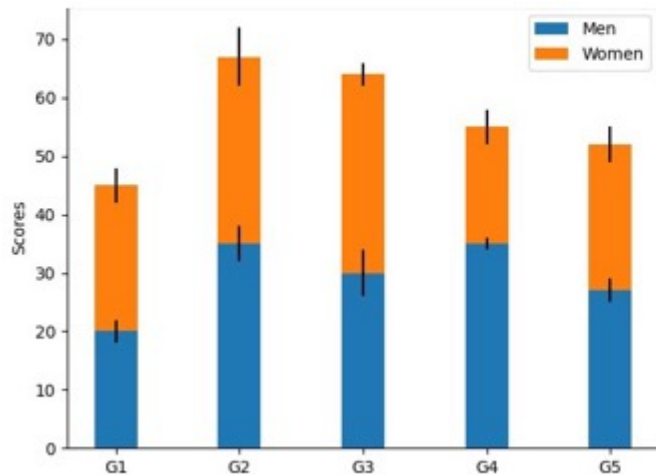


seaborn

Section 1 Matplotlib

Matplotlib

- Widely-used for basic plotting
- Highly customizable
- Works with NumPy and pandas



Matplotlib

Line Chart

Line chart is one of the basic plots and can be created using `plot()` function. It is used to represent a relationship between two data X and Y on a different axis.

```
import matplotlib.pyplot as plt
```

```
x = [10, 20, 30, 40]
```

```
y = [20, 25, 35, 55]
```

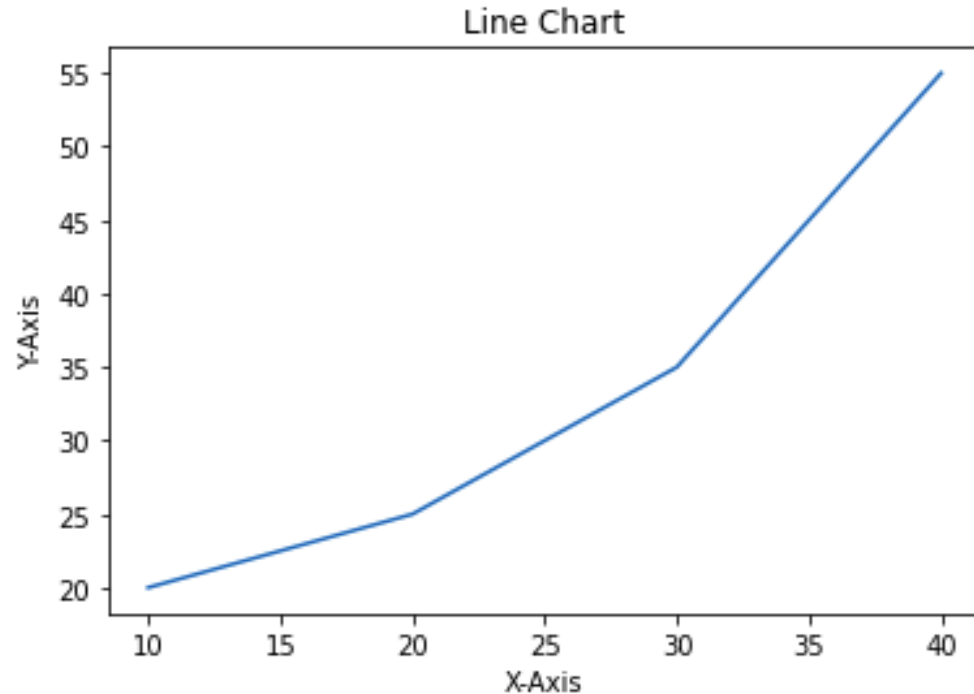
```
plt.plot(x, y)
```

```
plt.title("Line Chart")
```

```
plt.ylabel('Y-Axis')
```

```
plt.xlabel('X-Axis')
```

```
plt.show()
```



Matplotlib

Bar Chart

Bar chart displays categorical data using rectangular bars whose lengths are proportional to the values they represent. It can be plotted vertically or horizontally to compare different categories.

```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun']
```

```
y = [170, 120, 250, 190]
```

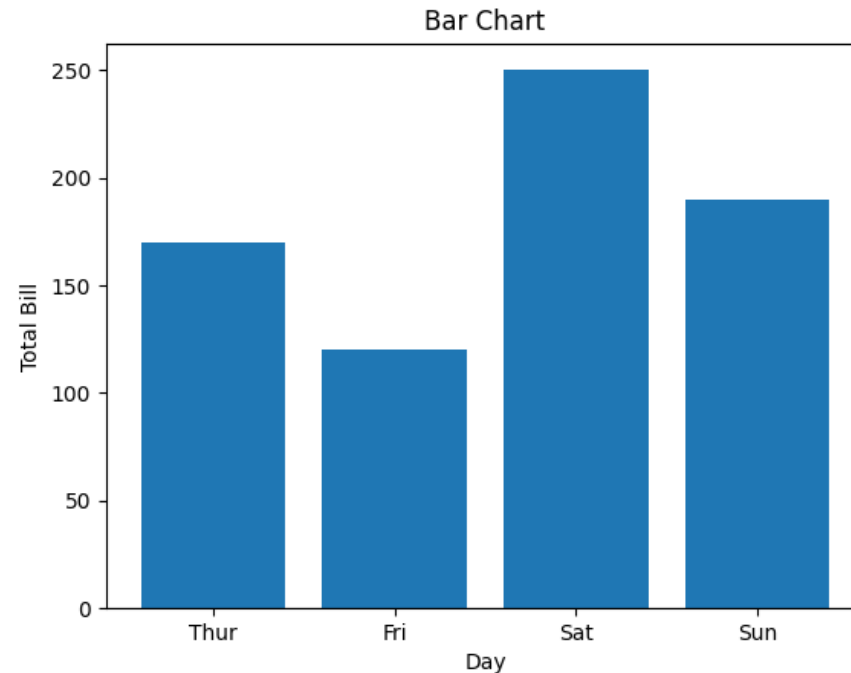
```
plt.bar(x, y)
```

```
plt.title("Bar Chart")
```

```
plt.xlabel("Day")
```

```
plt.ylabel("Total Bill")
```

```
plt.show()
```



Matplotlib

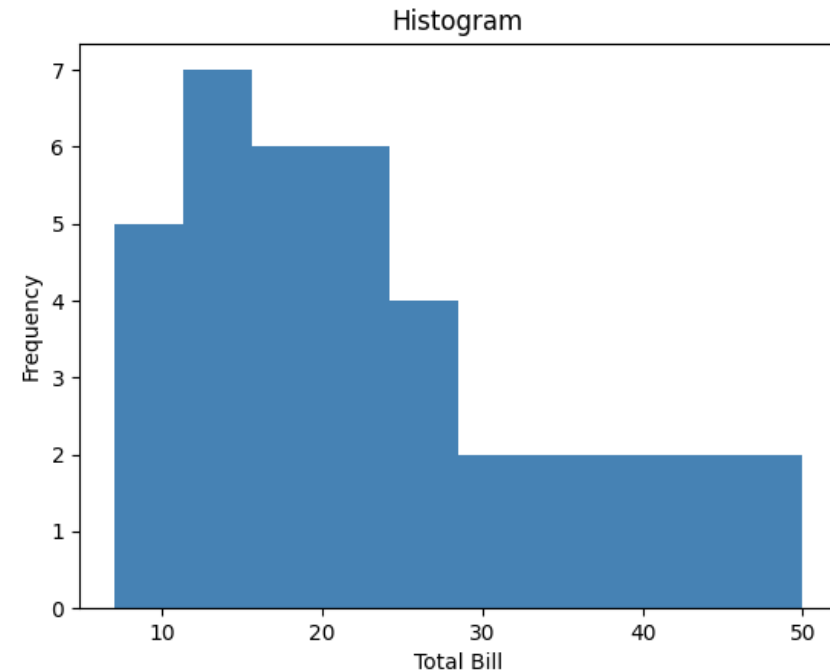
Histogram

Histogram shows the **distribution** of data by grouping values into bins. The `hist()` function is used to create it, with X-axis showing bins and Y-axis showing frequencies.

```
import matplotlib.pyplot as plt

x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14,
     14, 15, 16, 16, 17, 18, 18, 19, 20, 20,
     21, 22, 23, 24, 25, 25, 26, 28, 30,
     32, 35, 36, 38, 40, 42, 44, 48, 50]

plt.hist(x, bins=10, color='steelblue')
plt.title("Histogram")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```



Matplotlib

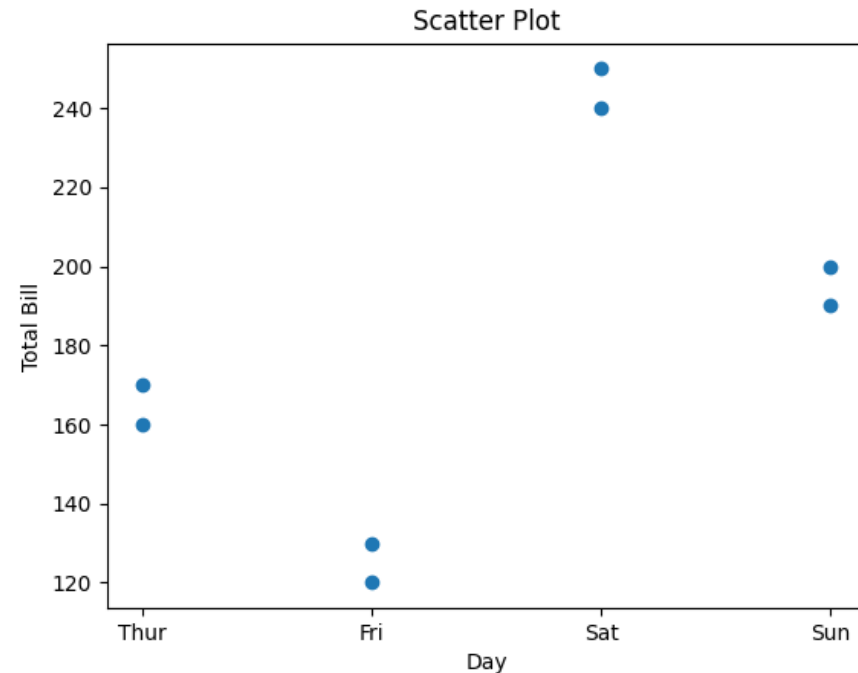
Scatter Plot

Scatter plots are used to observe relationships between variables. The `scatter()` method in the `matplotlib` library is used to draw a scatter plot.

```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur',
     'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190, 160, 130, 240,
     200]

plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```



Matplotlib

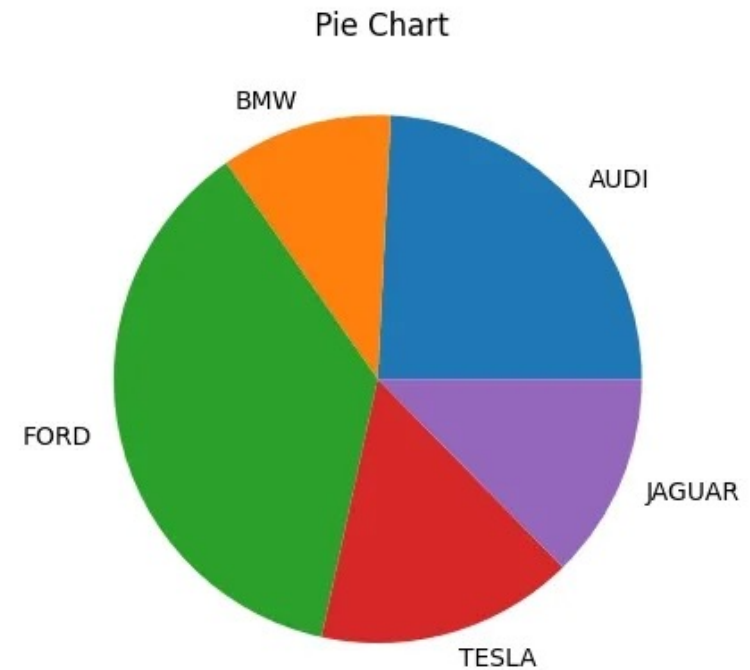
Pie Chart

Pie chart is a circular chart used to show data as **proportions** or **percentages**. It is created using the `pie()`, where each slice (wedge) represents a part of the whole.

```
import matplotlib.pyplot as plt
import pandas as pd

cars = ['AUDI', 'BMW', 'FORD', 'TESLA',
        'JAGUAR',]
data = [23, 10, 35, 15, 12]

plt.pie(data, labels=cars)
plt.title(" Pie Chart")
plt.show()
```



Matplotlib

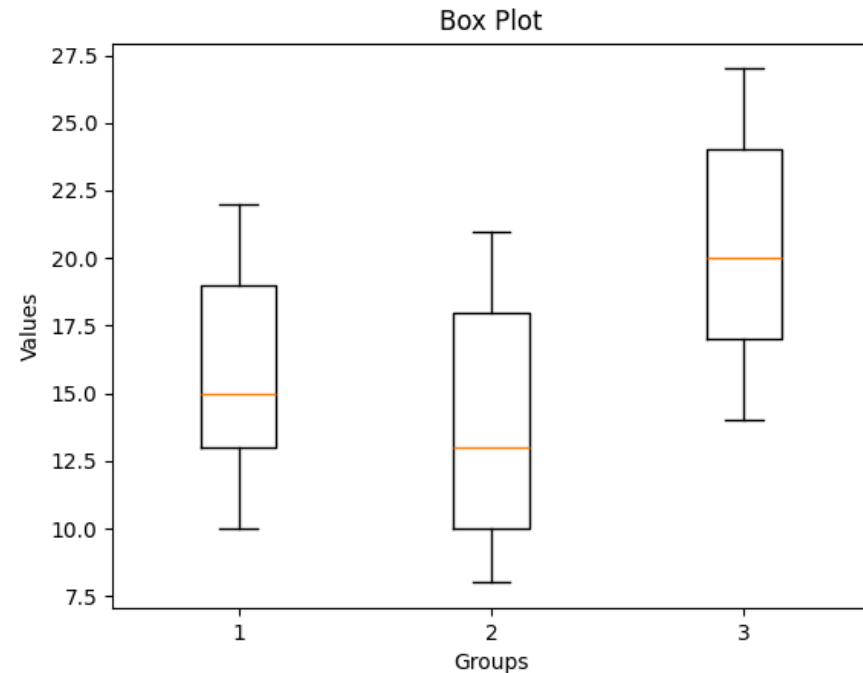
Box Plot

Box plot is a simple graph that shows how data is spread out. It displays the **minimum**, **maximum**, **median** and **quartiles** and also helps to spot outliers easily.

```
import matplotlib.pyplot as plt

data = [ [10, 12, 14, 15, 18, 20, 22],
         [8, 9, 11, 13, 17, 19, 21],
         [14, 16, 18, 20, 23, 25, 27] ]

plt.boxplot(data)
plt.xlabel("Groups")
plt.ylabel("Values")
plt.title("Box Plot")
plt.show()
```



Matplotlib

Heatmap

Heatmap is a graphical representation of data where values are shown as colors. It helps visualize **patterns, correlations** or **intensity** in a **matrix-like format**. It is created using `imshow()` method in Matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
np.random.seed(0)
data = np.random.rand(10, 10)
```

```
plt.imshow(data, cmap='viridis',
            interpolation='nearest')
```

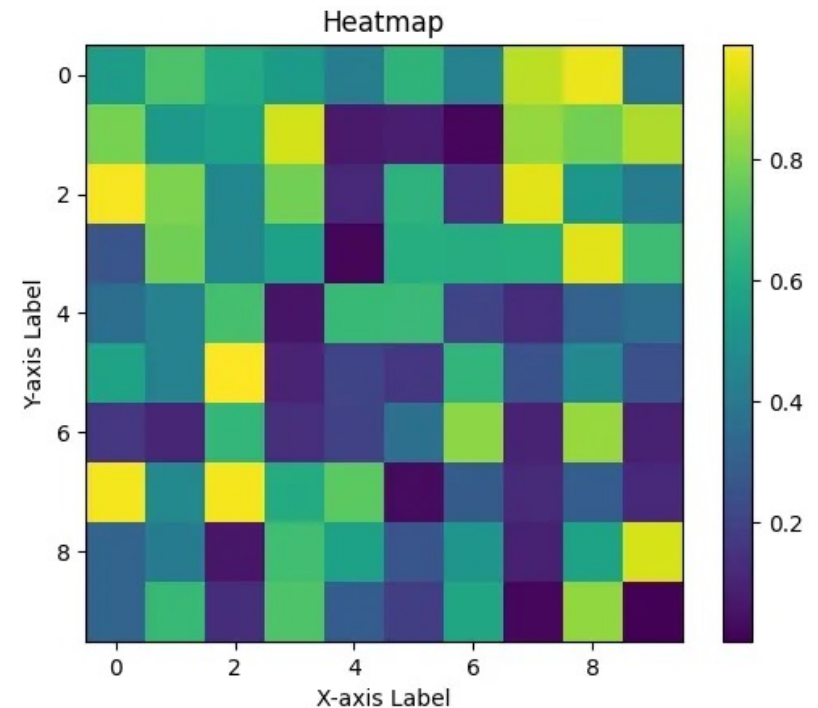
```
plt.colorbar()
```

```
plt.xlabel('X-axis Label')
```

```
plt.ylabel('Y-axis Label')
```

```
plt.title('Heatmap')
```

```
plt.show()
```



How to Customize Matplotlib Visualizations?

Customization in Matplotlib allows us to improve look and clarity of plots by adjusting elements like colors, styles, labels, titles and gridlines.

It helps make visualizations more informative and visually appealing for better data communication.

Matplotlib

Customizing Line Chart

Line charts can be customized using various properties:

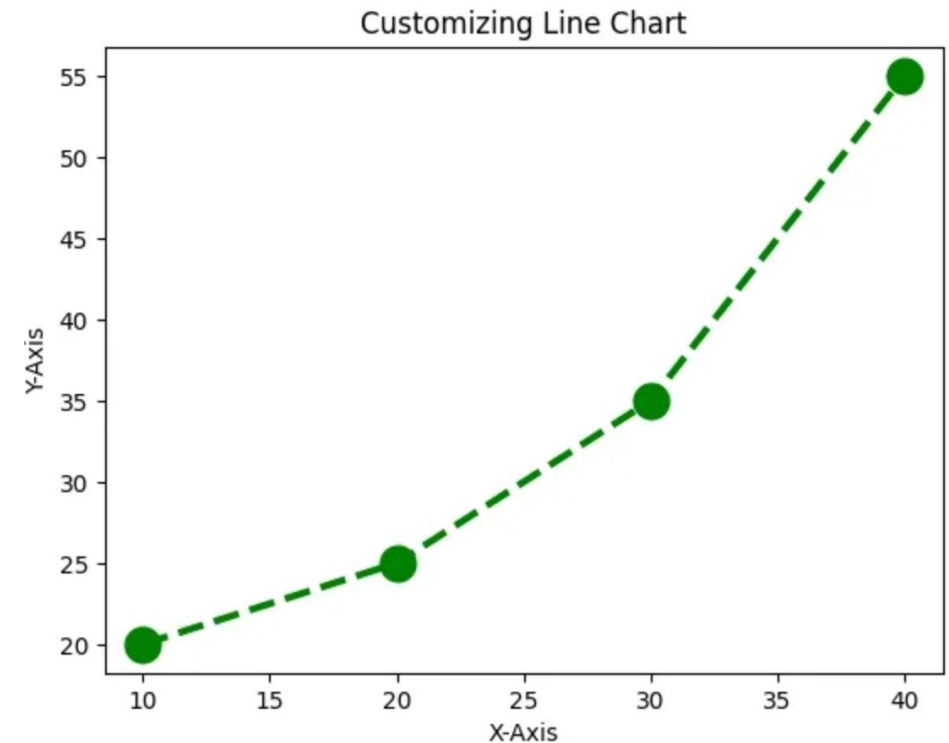
- **Color:** Change the color of the line
- **Linewidth:** Adjust the width of the line
- **Marker:** Change the style of plotted points
- **Markersize:** Change the size of the markers
- **Linestyle:** Define the style of the line like solid, dashed, etc.

```
import matplotlib.pyplot as plt

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

plt.plot(x, y, color='green', linewidth=3,
marker='o', markersize=15, linestyle='--')

plt.title("Customizing Line Chart")
plt.ylabel('Y-Axis')
plt.xlabel('X-Axis')
plt.show()
```



Matplotlib

Customizing Bar Chart

Bar charts can be made more informative and visually appealing by customizing:

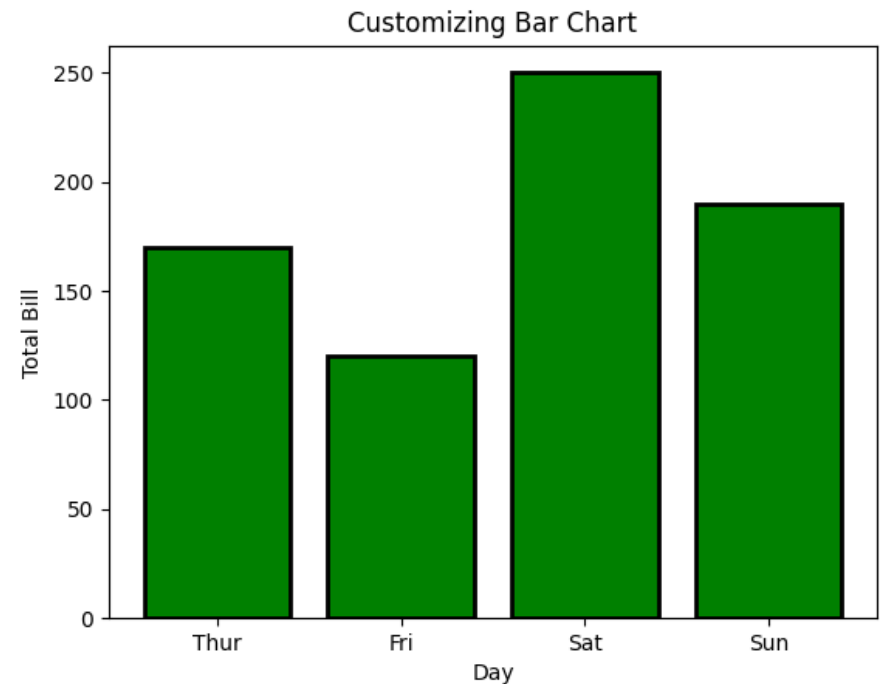
- **Color:** Fill color of the bars
- **Edgecolor:** Color of the bar edges
- **Linewidth:** Thickness of the edges
- **Width:** Width of each bar

```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]

plt.bar(x, y, color='green',
        edgecolor='black', linewidth=2)

plt.title("Customizing Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```



Matplotlib

Customizing Histogram Plot

To make histogram plots more effective various customizations can be applied:

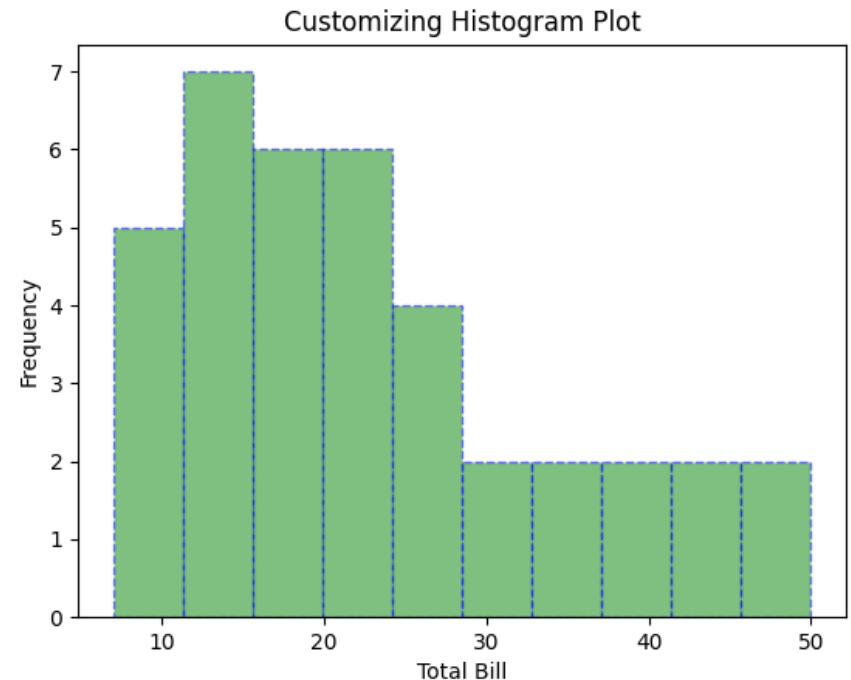
- **Bins:** Number of groups (bins) to divide data into
- **Color:** Bar fill color
- **Edgecolor:** Bar edge color
- **Linestyle:** Style of the edges like solid, dashed, etc.
- **Alpha:** Transparency level (0 = transparent, 1 = opaque)

```
import matplotlib.pyplot as plt
```

```
x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14,  
     15, 16, 16, 17,  
     18, 18, 19, 20, 20, 21, 22, 23, 24, 25,  
     25, 26, 28, 30,  
     32, 35, 36, 38, 40, 42, 44, 48, 50]
```

```
plt.hist(x, bins=10, color='green',  
edgecolor='blue',linestyle='--', alpha=0.5)
```

```
plt.title("Customizing Histogram Plot")  
plt.xlabel("Total Bill")  
plt.ylabel("Frequency")  
plt.show()
```



Customizing Scatter Plot

Scatter plots can be enhanced with:

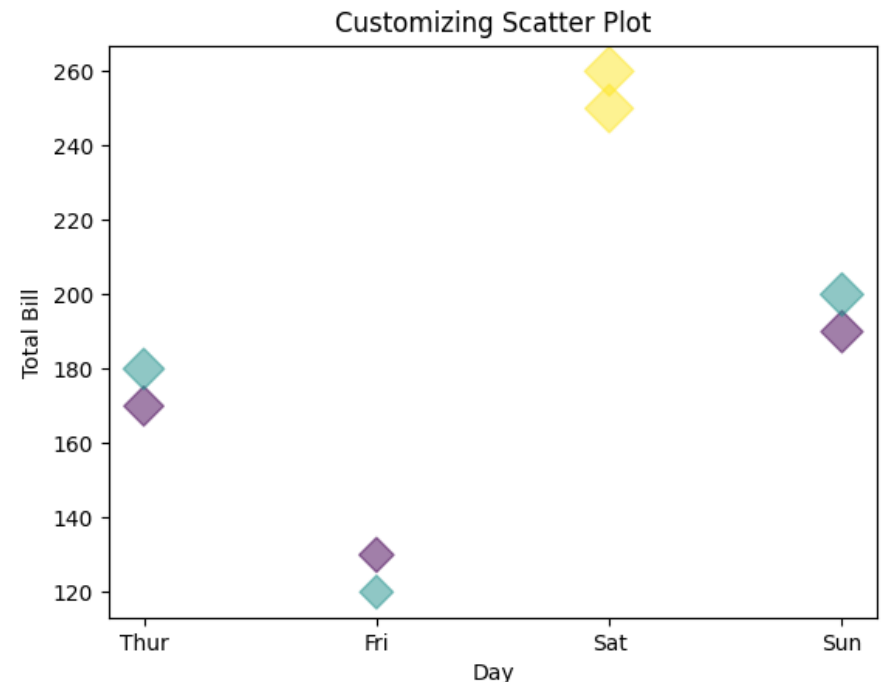
- **S:** Marker size (single value or array)
- **C:** Color of markers or sequence of colors
- **Marker:** Marker style like circle, diamond, etc.
- **Linewidths:** Width of marker borders
- **Edgecolor:** Color of marker borders
- **Alpha:** Blending value, between 0 (transparent) and 1 (opaque)

```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri',  
     'Sat', 'Sun']  
y = [170, 120, 250, 190, 180, 130, 260, 200]  
size = [2, 3, 4, 2, 3, 2, 4, 3]  
bill = [170, 120, 250, 190, 180, 130, 260, 200]
```

```
plt.scatter(x, y, c=size, s=bill, marker='D',  
           alpha=0.5)
```

```
plt.title("Customizing Scatter Plot")  
plt.xlabel("Day")  
plt.ylabel("Total Bill")  
plt.show()
```



Matplotlib

Customizing Pie Chart

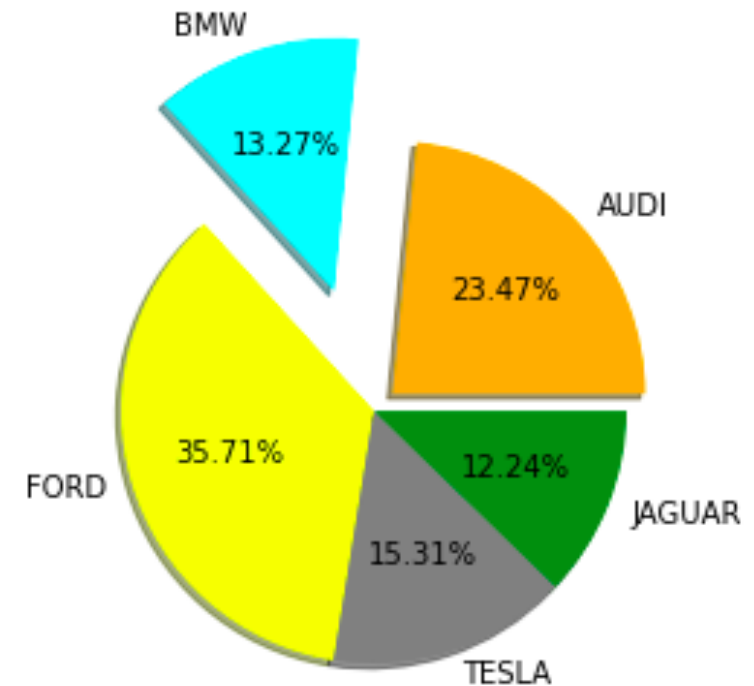
To make pie charts more effective and visually appealing use:

- **Explode:** Moving the wedges of the plot
- **Color:** Colors of the slices
- **Autopct:** Label the wedge with their numerical value
- **Shadow:** Used to create a shadow effect

```
import matplotlib.pyplot as plt  
import pandas as pd
```

```
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR']  
data = [23, 13, 35, 15, 12]  
explode = [0.1, 0.5, 0, 0, 0]  
colors = ("orange", "cyan", "yellow", "grey", "green")
```

```
plt.pie(data, labels=cars, explode=explode,  
autopct='%1.2f%%', colors=colors, shadow=True)  
plt.show()
```



Matplotlib

Matplotlib's Core Components: Figures

The Figure class represents the full drawing area or canvas that can hold one or more plots. It is created using the `figure()` function and lets user control the overall size, layout and background of the plot window.

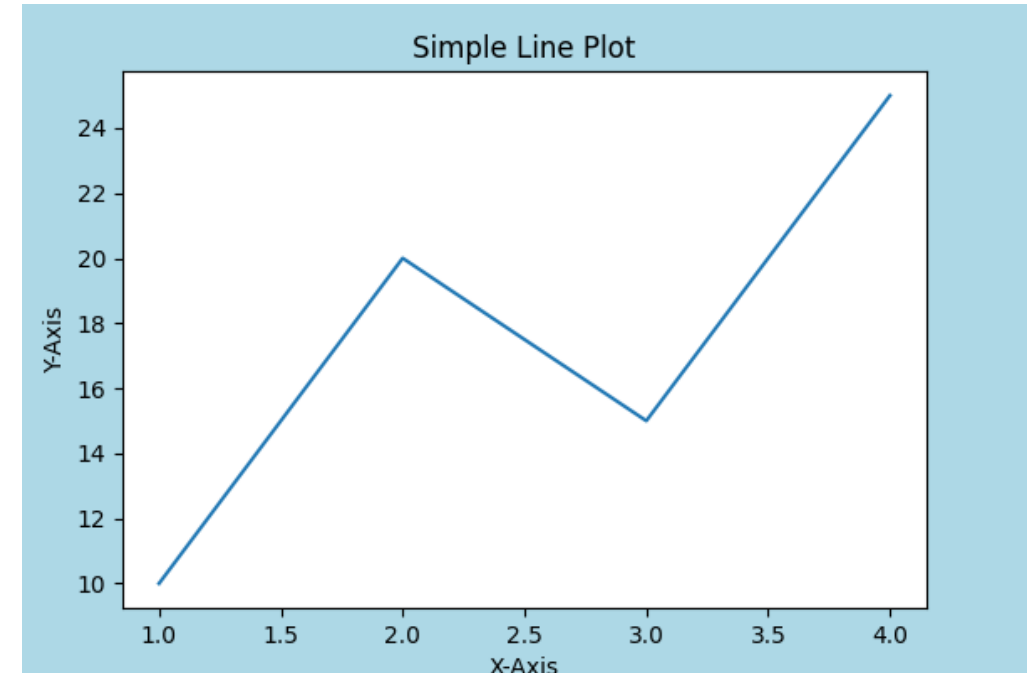
- **figsize:** Sets size of the figure (width, height) in inches.
- **facecolor:** Sets background color of the figure.

```
import matplotlib.pyplot as plt

# Create a Figure with basic size and background color
fig = plt.figure(figsize=(6, 4), facecolor='lightblue')

# Add Axes to the Figure
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
ax.plot(x, y)

plt.title("Simple Line Plot")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.show()
```



Matplotlib

Matplotlib's Core Components: Figures

The `subplot()` method adds a plot to a specified grid position within the current figure. It takes three arguments:

- the number of rows
- rows columns
- plot index

```
import matplotlib.pyplot as plt
```

```
x = [10, 20, 30, 40]
```

```
y = [20, 25, 35, 55]
```

```
plt.figure()
```

```
plt.subplot(121)
```

```
plt.plot(x, y)
```

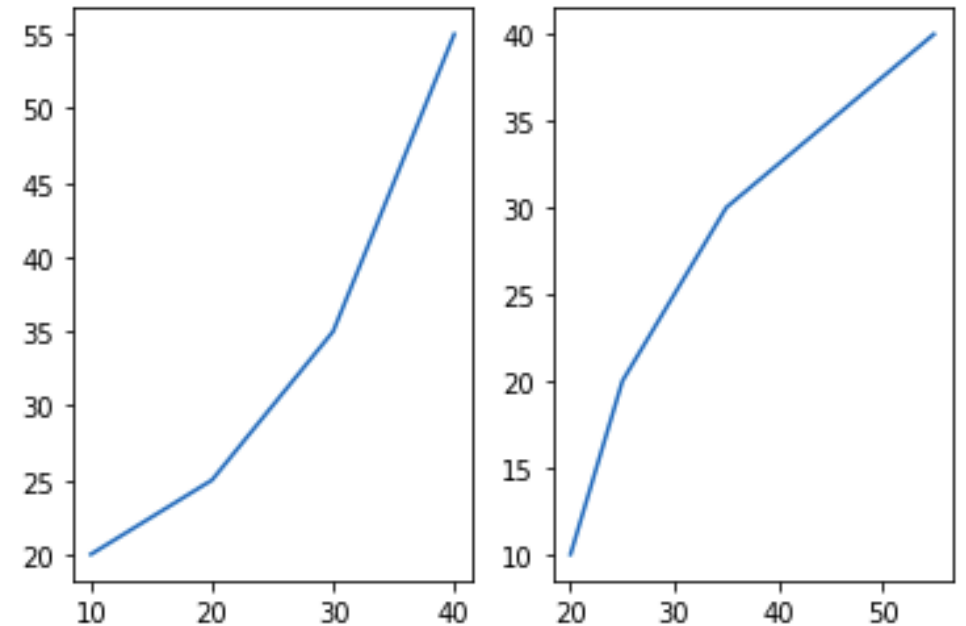
```
plt.subplot(122)
```

```
plt.plot(y, x)
```

```
plt.show()
```

121 means:

- 1 row
- 2 columns
- The first figure



Matplotlib

Matplotlib's Core Components: Axes

Axes class represents actual plotting area where data is drawn. It is the most basic and flexible for creating plots or subplots within a figure. A single figure can contain multiple axes but each Axes object belongs to only one figure. It can create an Axes object using `axes()` function.

```
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

fig = plt.figure(figsize=(5, 4))
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])

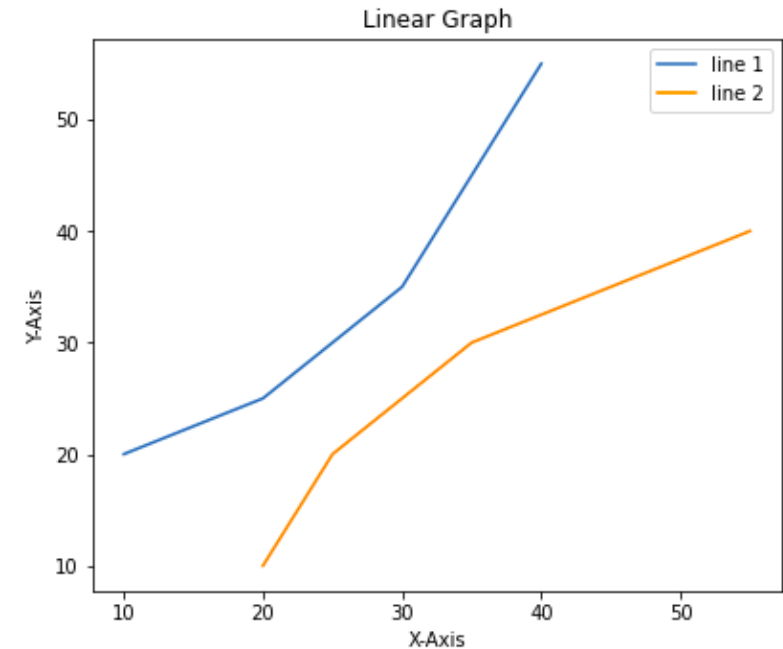
ax.plot(x, y)
ax.plot(y, x)

ax.set_title("Linear Graph")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
ax.legend(labels=('line 1', 'line 2'))
plt.show()
```

- `fig.add_axes()` manually adds axes
- `[0.1, 0.1, 0.8, 0.8]` defines the position left, bottom, width and height as a fraction of figure size.

• Set axel labels

• Set legend



Matplotlib

Saving Plots

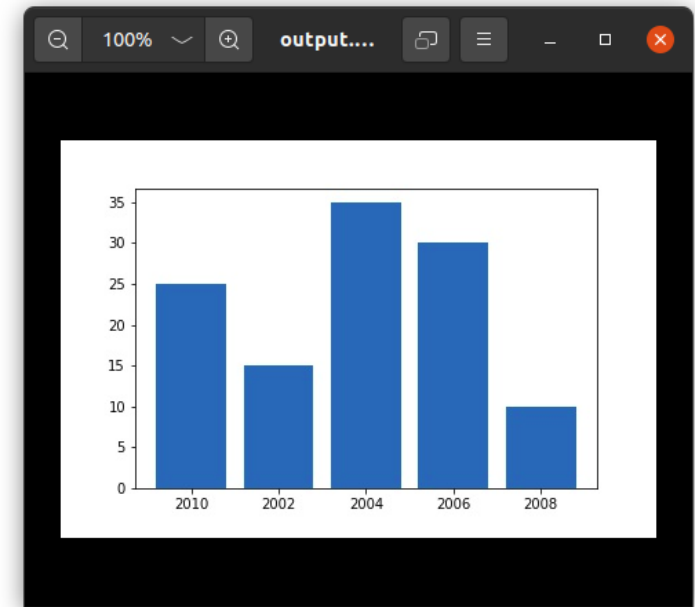
Matplotlib offers `savefig()` function to save the current plot to a file. By changing file extension, you can store the plot in different formats such as `.png`, `.jpg`, `.pdf` or `.svg`.

```
import matplotlib.pyplot as plt

year = ['2010', '2002', '2004', '2006',
        '2008']
production = [25, 15, 35, 30, 10]

plt.bar(year, production)

plt.savefig("output.jpg")
plt.savefig("output1", facecolor='y',
            bbox_inches="tight", pad_inches=0.3,
            transparent=True)
```

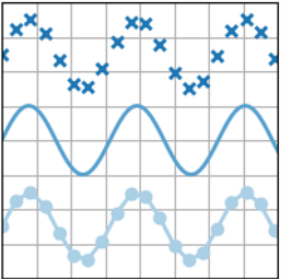


Matplotlib

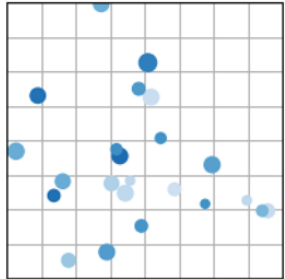
Full List of Plot Types

Pairwise data

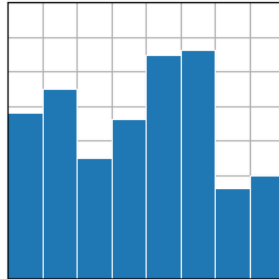
Plots of pairwise (x,y) , tabular $(\text{var}_0, \dots, \text{var}_n)$, and functional $f(x)=y$ data.



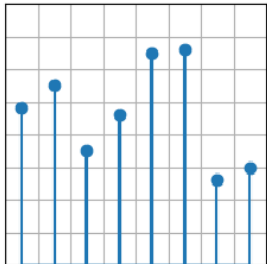
`plot(x, y)`



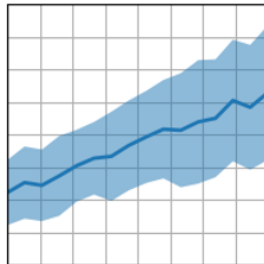
`scatter(x, y)`



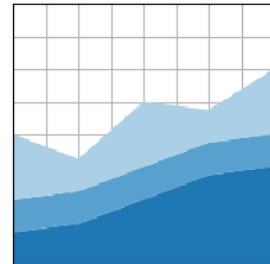
`bar(x, height)`



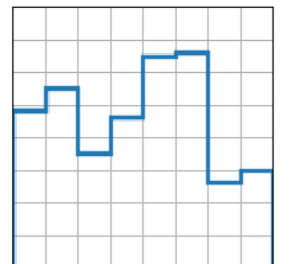
`stem(x, y)`



`fill_between(x, y1, y2)`



`stackplot(x, y)`



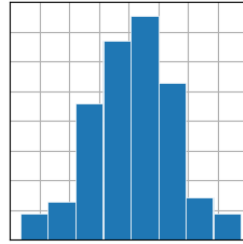
`stairs(values)`

Matplotlib

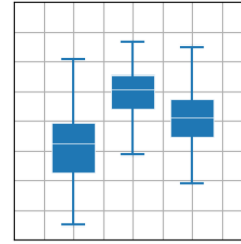
Full List of Plot Types

Statistical distributions

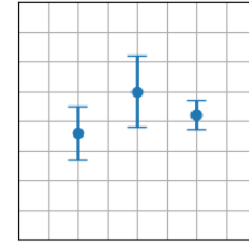
Plots of the distribution of at least one variable in a dataset. Some of these methods also compute the distributions.



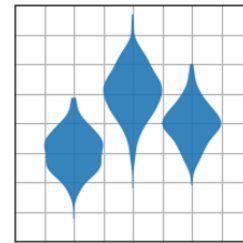
hist(x)



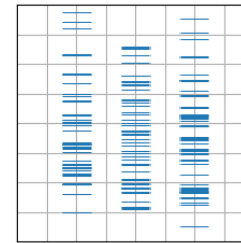
boxplot(X)



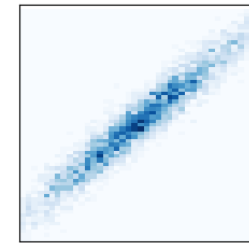
errorbar(x, y, yerr, xerr)



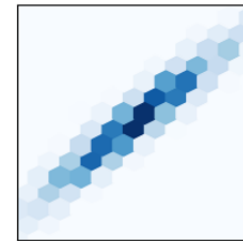
violinplot(D)



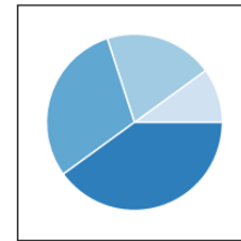
eventplot(D)



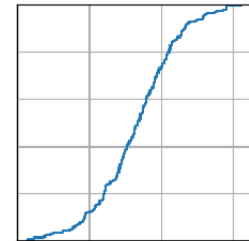
hist2d(x, y)



hexbin(x, y, C)



pie(x)

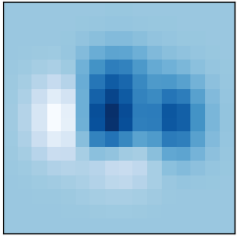


ecdf(x)

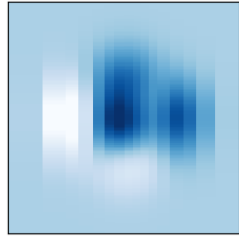
Full List of Plot Types

Gridded data

Plots of arrays and images Z_{ij} and fields U_{ij}, V_{ij} on regular grids and corresponding coordinate grids X_{ij}, Y_{ij} .



`imshow(Z)`



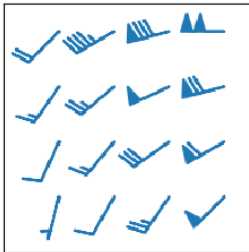
`pcolormesh(X, Y, Z)`



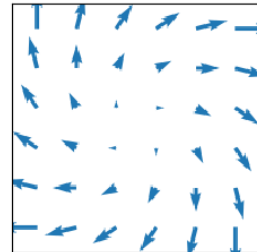
`contour(X, Y, Z)`



`contourf(X, Y, Z)`



`barbs(X, Y, U, V)`



`quiver(X, Y, U, V)`

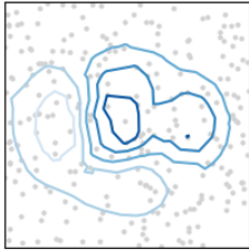


`streamplot(X, Y, U, V)`

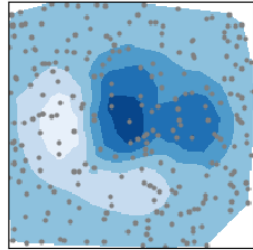
Full List of Plot Types

Irregularly gridded data

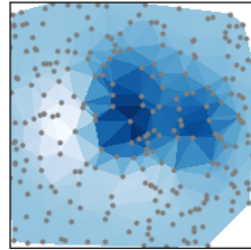
Plots of data $Z_{x,y}$ on unstructured grids, unstructured coordinate grids (x,y), and 2D functions $f(x,y)=z$.



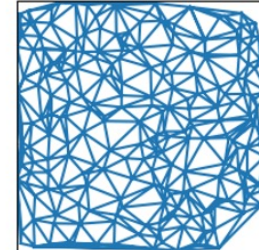
tricontour(x, y, z)



tricontourf(x, y, z)



tripcolor(x, y, z)



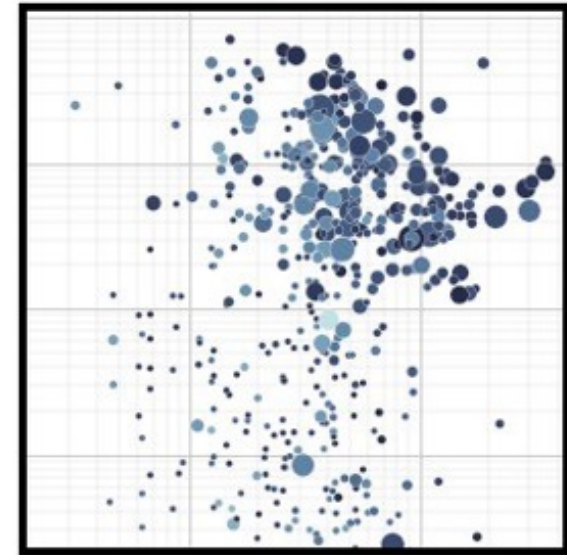
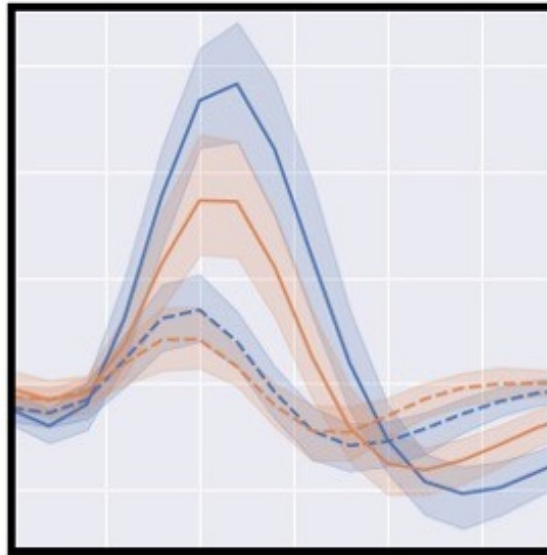
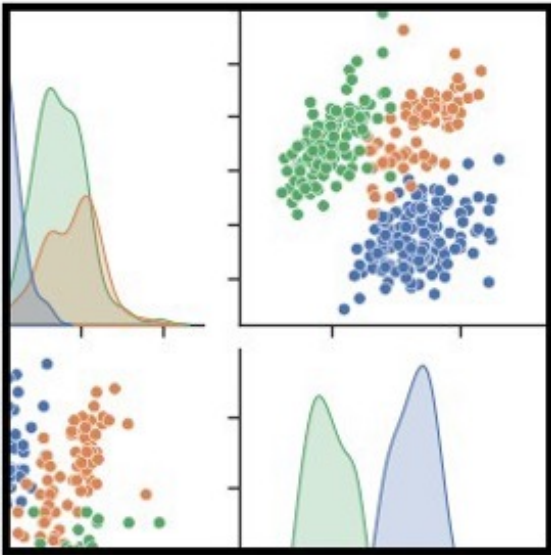
triplot(x, y)

Section 2 Seaborn

Seaborn

Why do we need seaborn when we already have matplotlib?

- Uses fewer syntax with good default themes
- Integrated to work great with pandas data-frame
- Uses Matplotlib under the hood (so we still need to know how Matplotlib works)



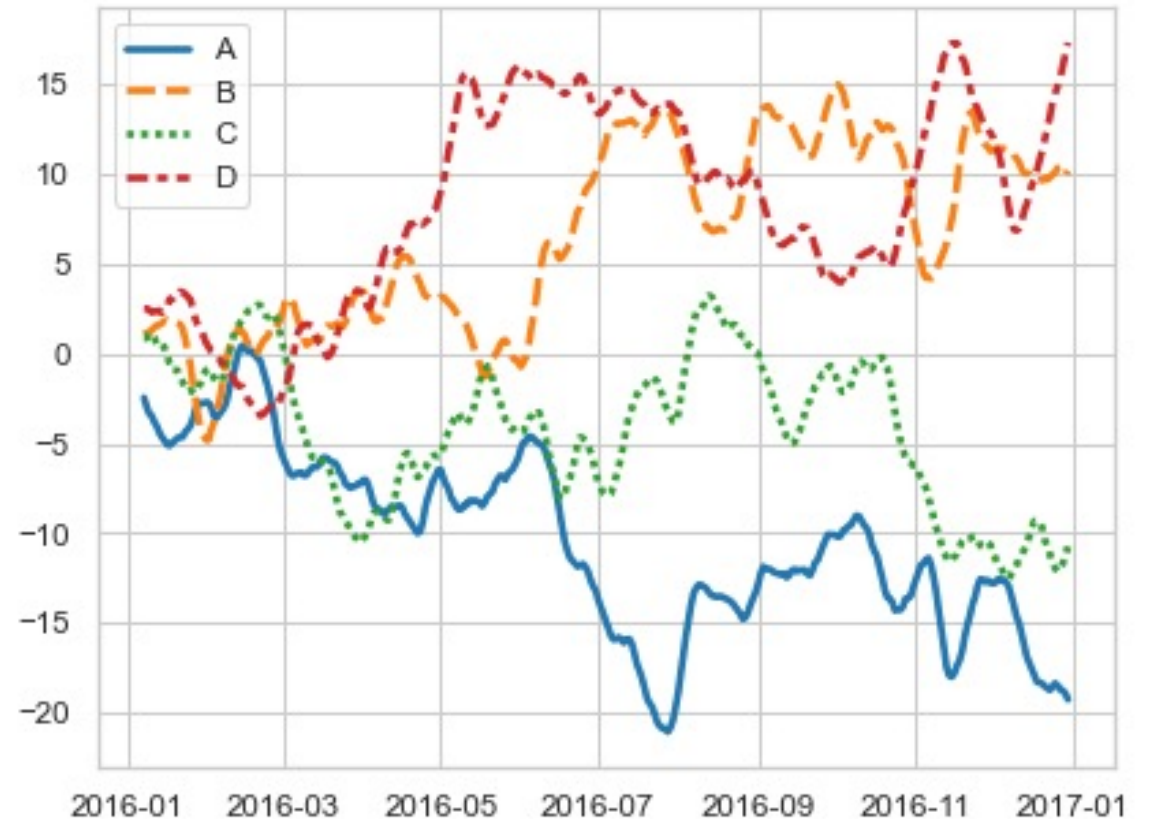
Seaborn

Seaborn Example (Line Chart)

```
import numpy as np
import pandas as pd
import seaborn as sns
sns.set_theme(style="whitegrid")

rs = np.random.RandomState(365)
values = rs.randn(365, 4).cumsum(axis=0)
dates = pd.date_range("1 1 2016",
                      periods=365, freq="D")
data = pd.DataFrame(values, dates,
                    columns=["A", "B", "C", "D"])
data = data.rolling(7).mean()

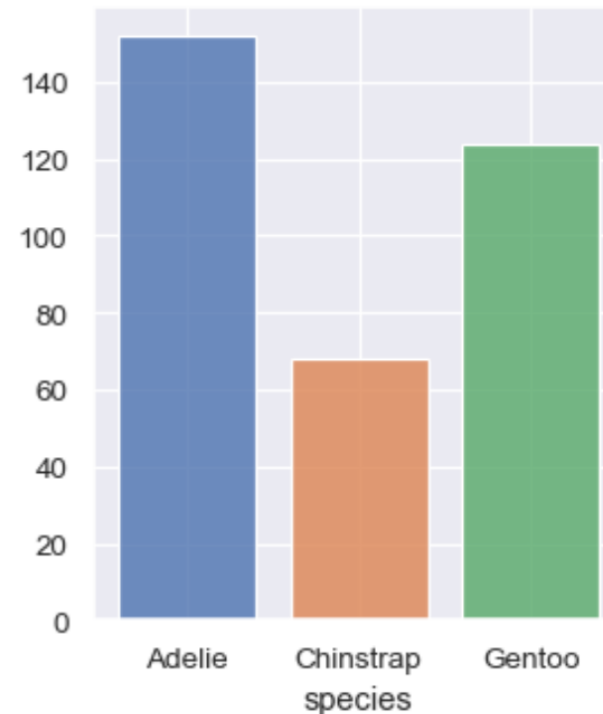
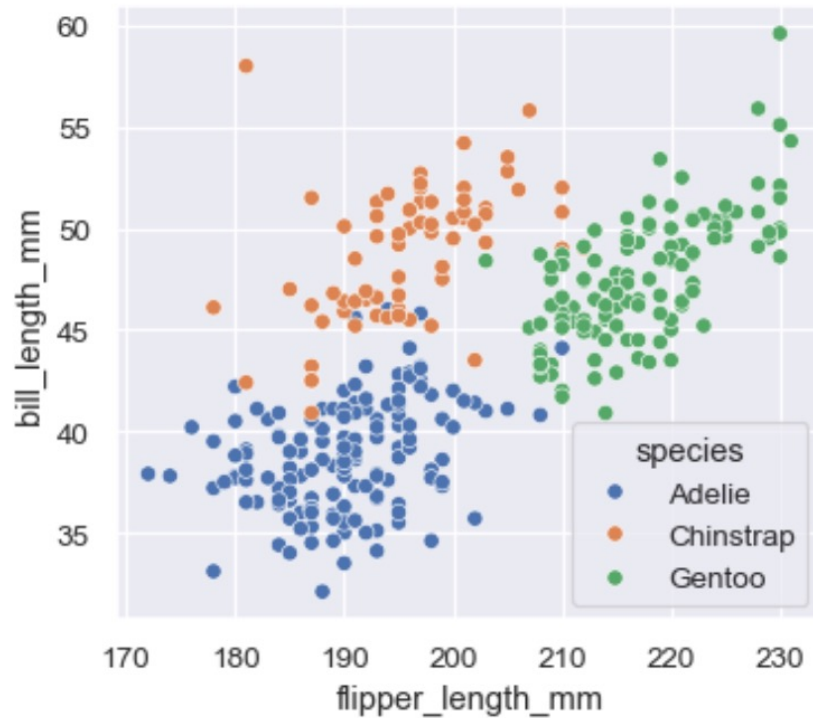
sns.lineplot(data=data, palette="tab10",
linewidth=2.5)
```



Seaborn

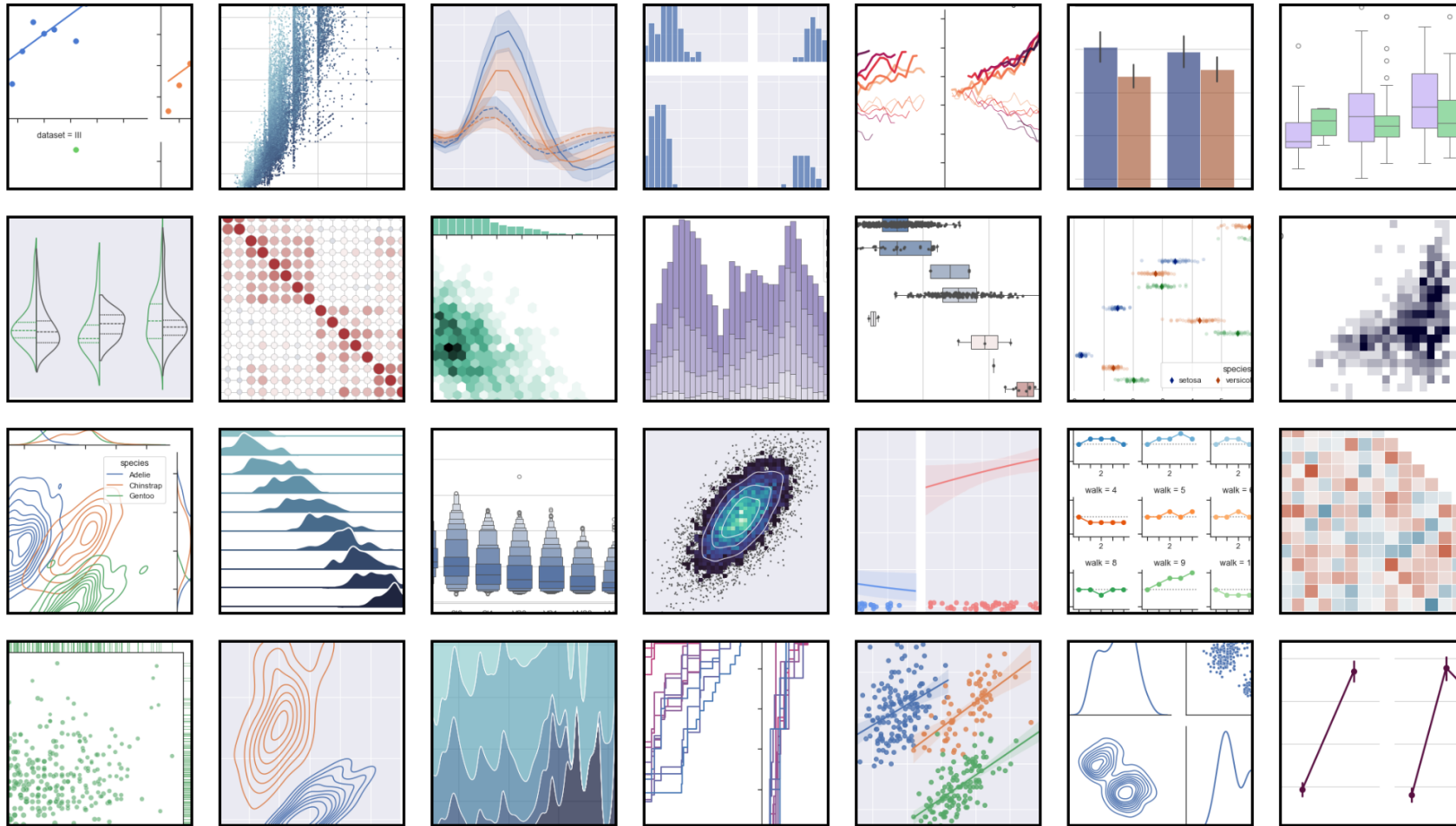
Similar Functions for Similar Plots (for most plots in Matplotlib)

```
penguins = sns.load_dataset("penguins")  
sns.scatterplot(data=penguins, x="flipper_length_mm", y="bill_length_mm",  
hue="species")  
sns.histplot(data=penguins, x="species", hue="species", shrink=.8, alpha=.8,  
legend=False)
```



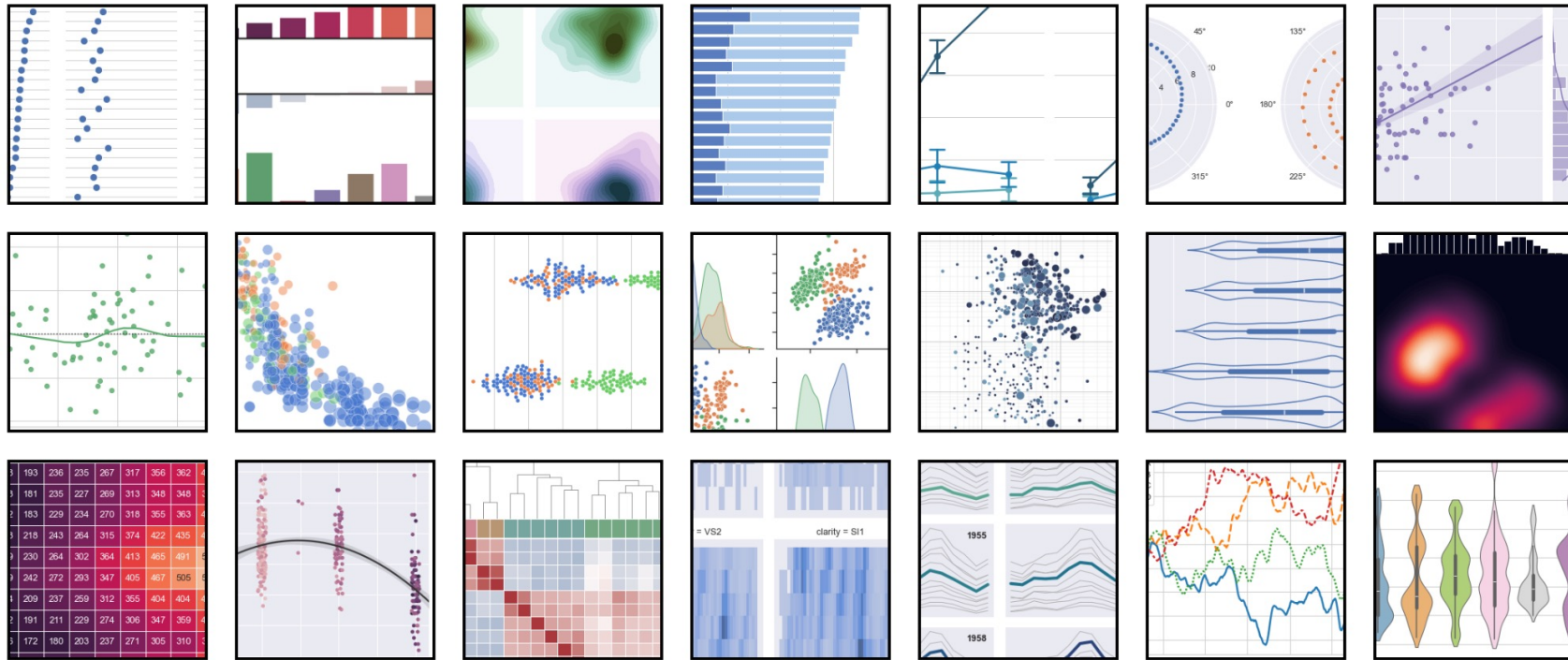
Seaborn

Example Gallery in Seaborn



Seaborn

Example Gallery in Seaborn



Seaborn

Useful Plots in Seaborn - Pair Plots

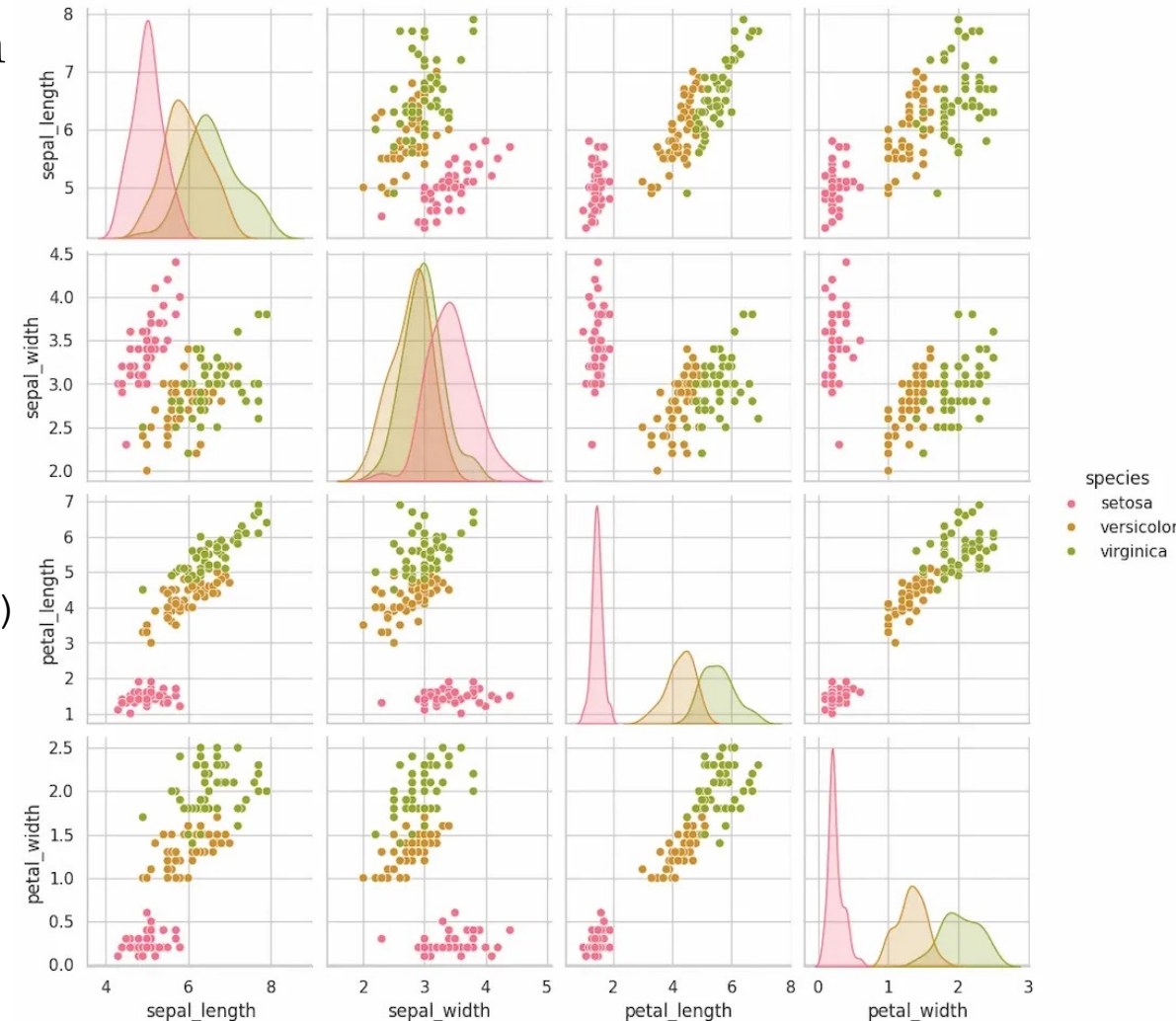
Pair plots are used to explore relationships between several variables by generating scatter plots for every pair of variables in a dataset along with univariate distributions on the diagonal.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.set_style("whitegrid")
custom_palette = sns.color_palette("husl", 8)
sns.set_palette(custom_palette)
```

```
data = sns.load_dataset("iris")
sns.pairplot(data, hue="species")
plt.show()
```

hue: (optional) Categorical variable used for color coding data points.



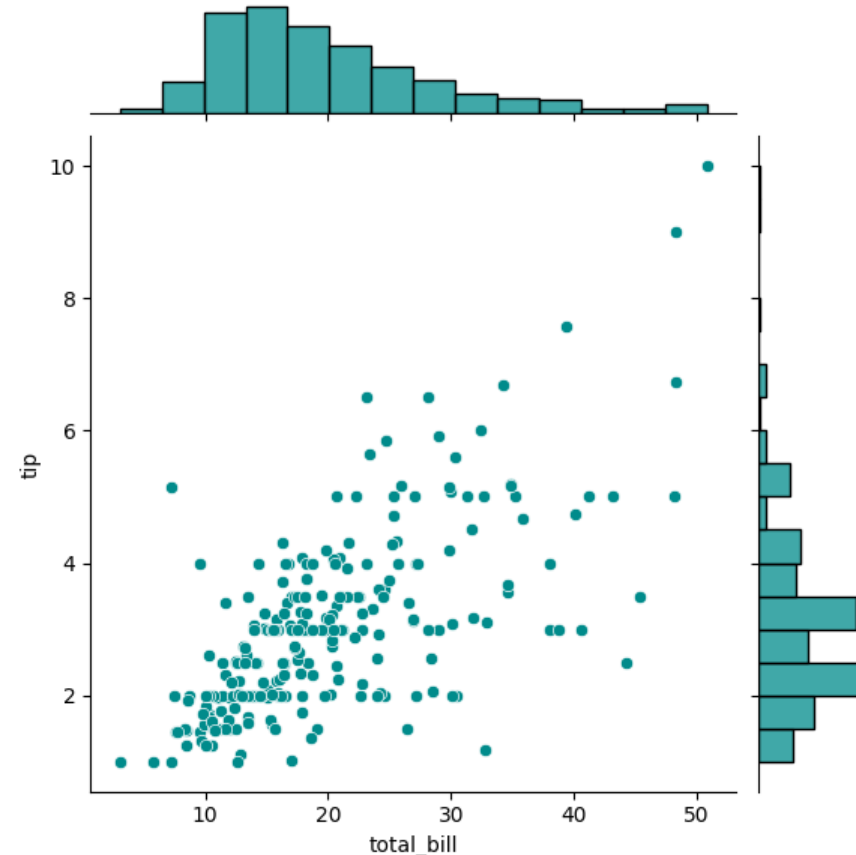
Seaborn

Useful Plots in Seaborn - Joint Plots

Joint plots combine a scatter plot with the distributions of the individual variables. This allows for a quick visual representation of how the variables are distributed individually and **how they relate to one another**.

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")
sns.jointplot(x="total_bill", y="tip",
data=data, kind="scatter",
color="#008B8B")
plt.show()
```



Seaborn

Useful Plots in Seaborn - Grid Plot

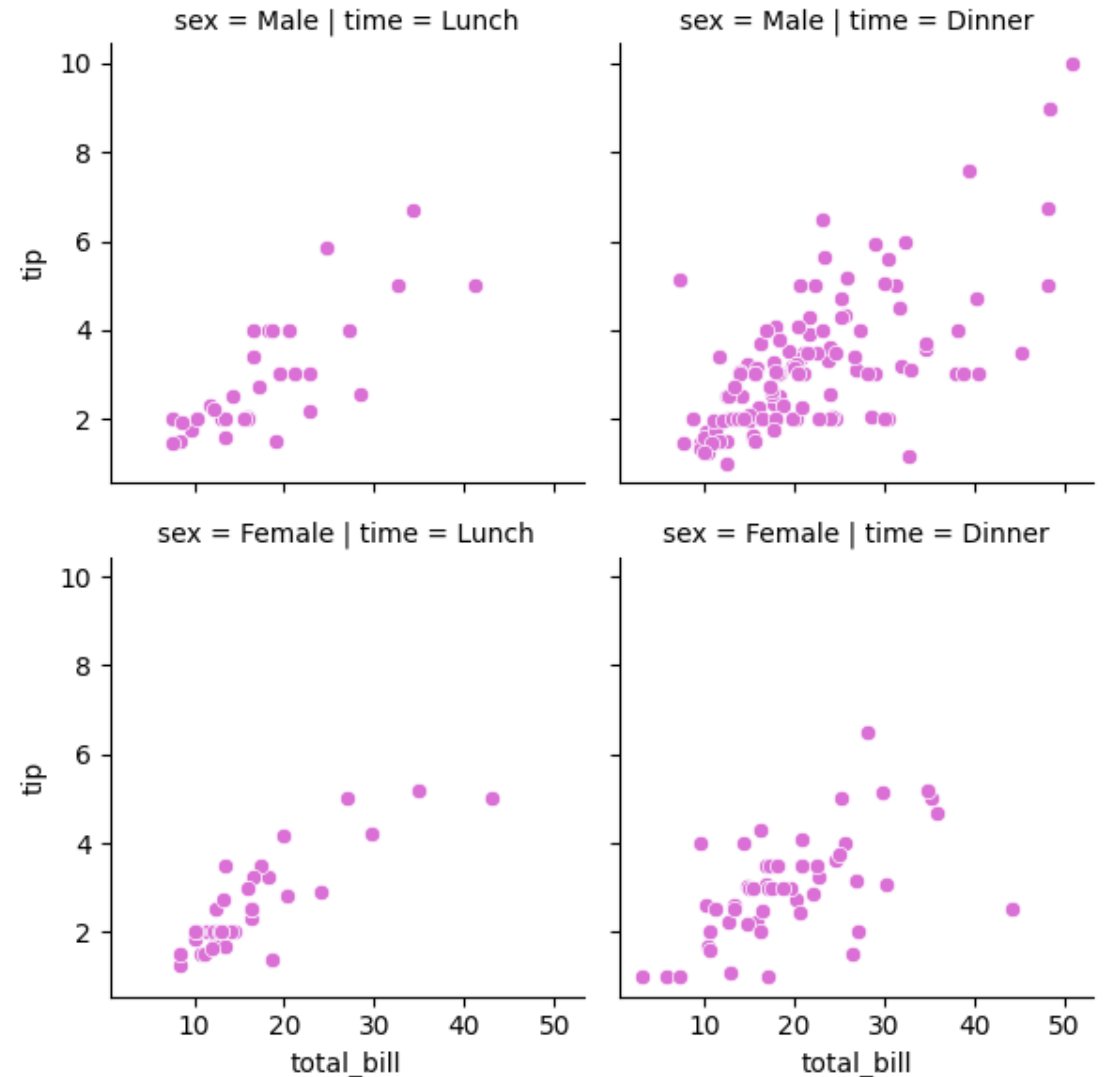
Grid plots in Seaborn are used to create multiple subplots in a grid layout. Using Seaborn's FacetGrid we can visualize **how variables interact across different categories** which makes it easier to compare groups or conditions within our dataset.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
tips = sns.load_dataset("tips")
```

```
plot=sns.FacetGrid(tips, col="time",
row="sex")
plot.map(sns.scatterplot,
"total_bill", "tip")
plt.show()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4



Seaborn

Useful Plots in Seaborn - Regression Plots

Note, regplot and lmlplot only support simple univariate regression ($y \sim x$) and cannot directly support multiple regression models.

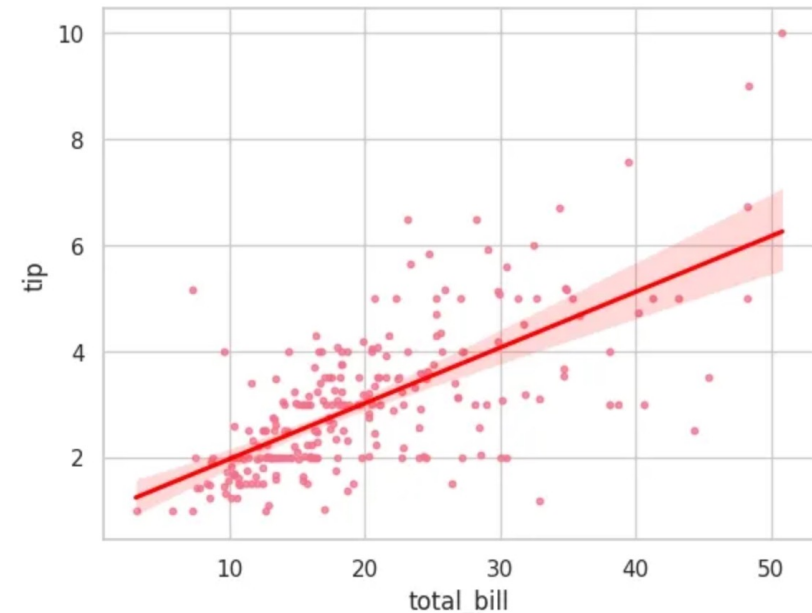
Seaborn simplifies the process of performing and visualizing regressions specifically linear regressions which is important for identifying relationships between variables, detecting trends and making predictions.

- `regplot()`: This function plots a scatter plot along with a linear regression model fit.
- `lmlplot()`: This function also plots linear models but provides more flexibility in handling multiple facets and datasets.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
tips = sns.load_dataset('tips')
```

```
sns.regplot(x='total_bill', y='tip',
data=tips, scatter_kws={'s':10},
line_kws={'color':'red'})
plt.show()
```



Section 3 Building Better Visualizations

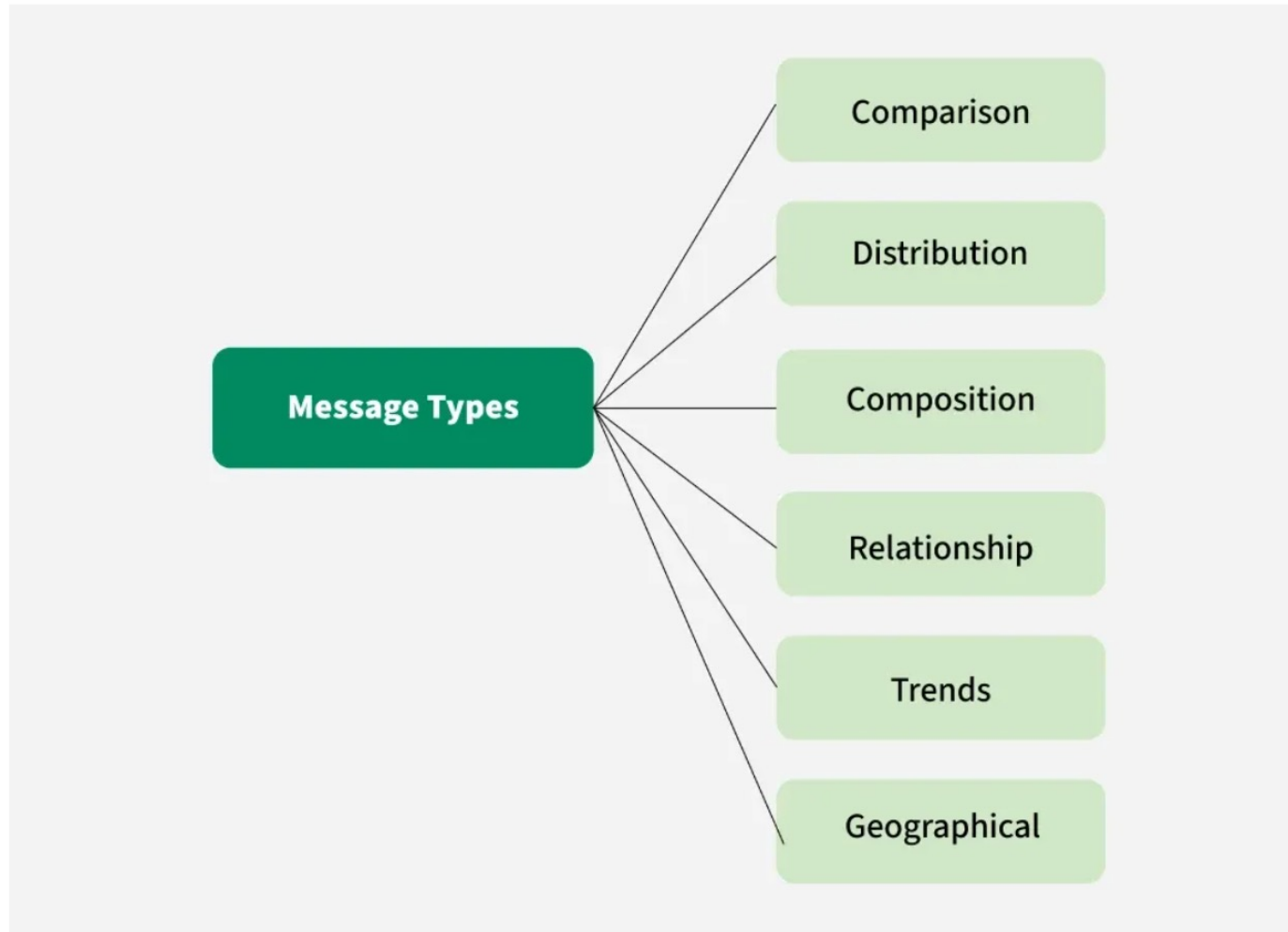
Choosing the Right Chart Type

Before choosing the right chart it's essential to understand the nature of your data and the message you want to convey. To understand consider the following questions:

1. What is the primary purpose of the visualization?
2. What type of data are you working with?
3. How many variables are you visualizing?
4. What is the volume of data points?

Criteria for Selecting Chart Types

The right chart depends on your data type and the message you want to communicate. Different charts serve different purposes:

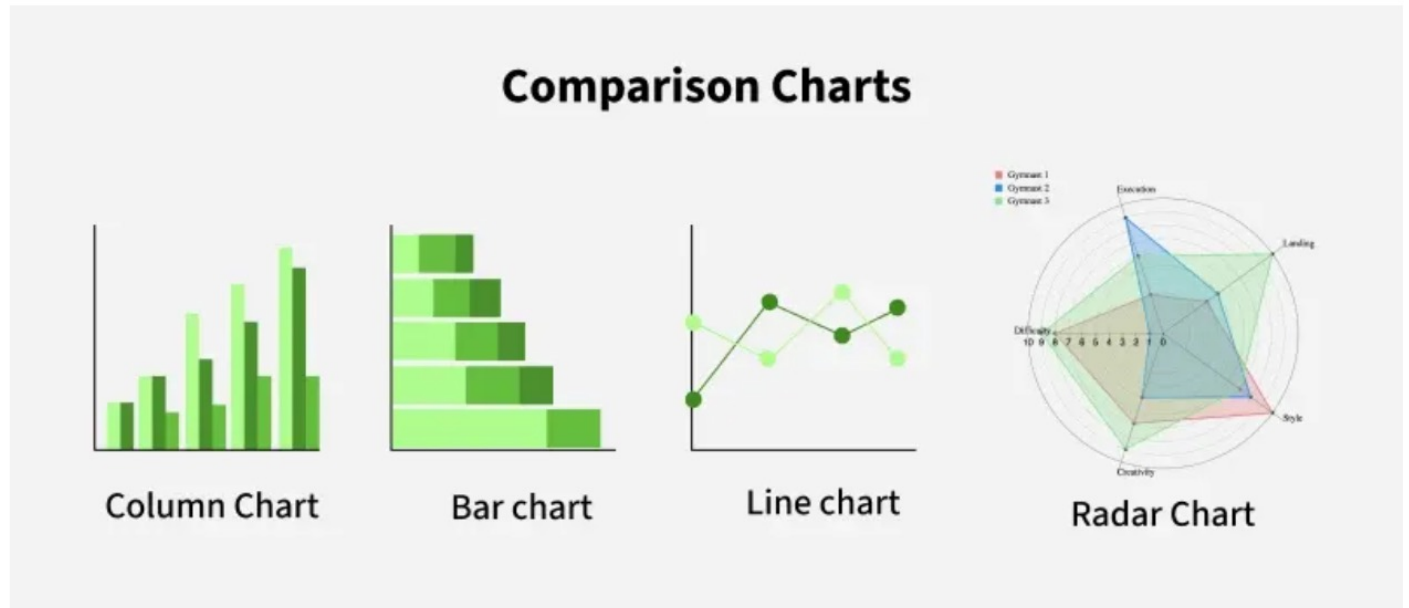


Comparison Charts

Comparison charts help visualize differences, patterns or trends between categories. They make it easy to compare values like sales, performance or populations.

Common types include:

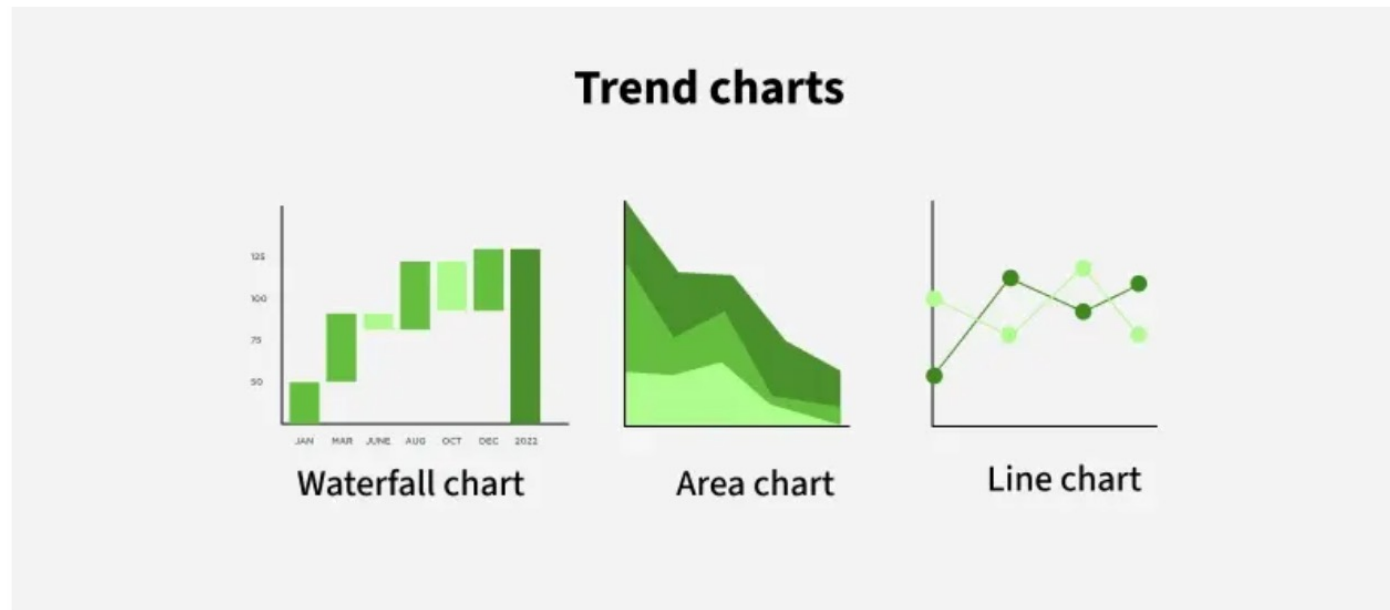
- **Column Chart:** Uses vertical bars, great for time-based or hierarchical data.
- **Bar Chart:** Compares categories using horizontal bars (e.g., product sales).
- **Line Chart:** Great for comparing trends by time.
- **Radar Chart:** Displays multiple variables in a circular layout, ideal for comparing performance across categories.



Trend Charts

Trend charts show how data changes over time, helping to spot patterns and movement. They are ideal for tracking progress, performance or change. Common types include:

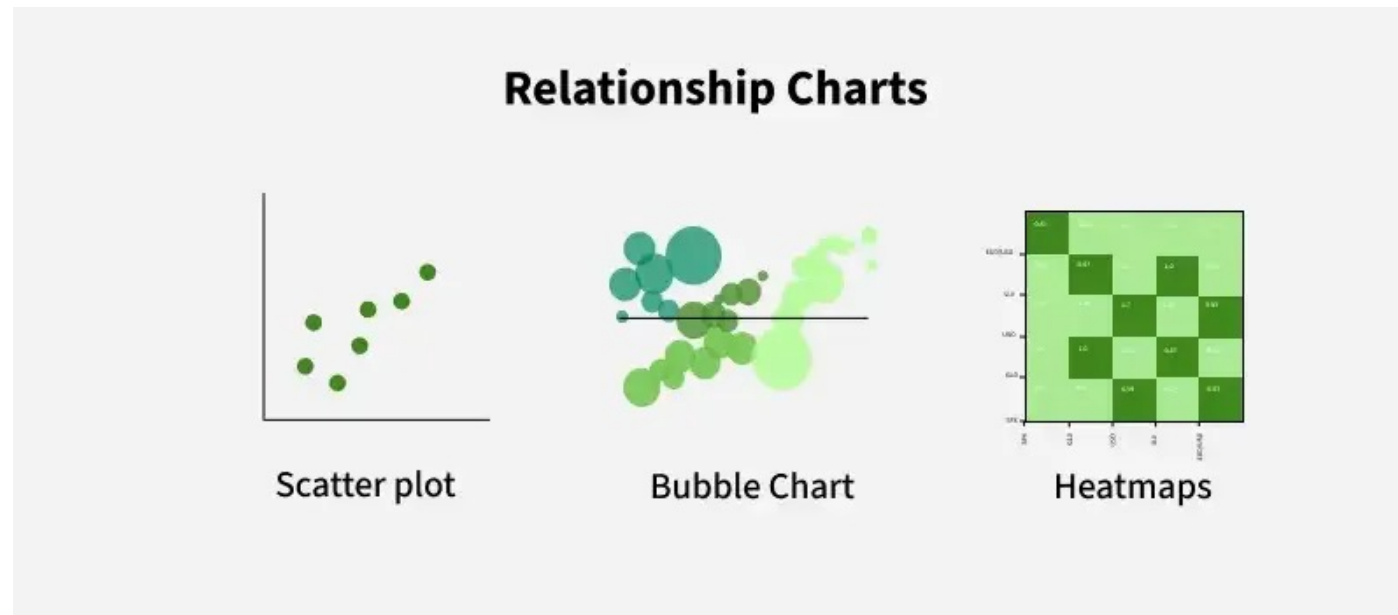
- **Line Chart:** Shows trends over time with connected points (e.g., monthly sales).
- **Area Chart:** Like a line chart but fills the space below, showing cumulative totals.
- **Waterfall Chart:** Displays how increases and decreases affect a total (e.g., profit changes).



Relationship Charts

Relationship charts show how variables are connected or correlated. They're useful for spotting patterns and understanding interdependencies.

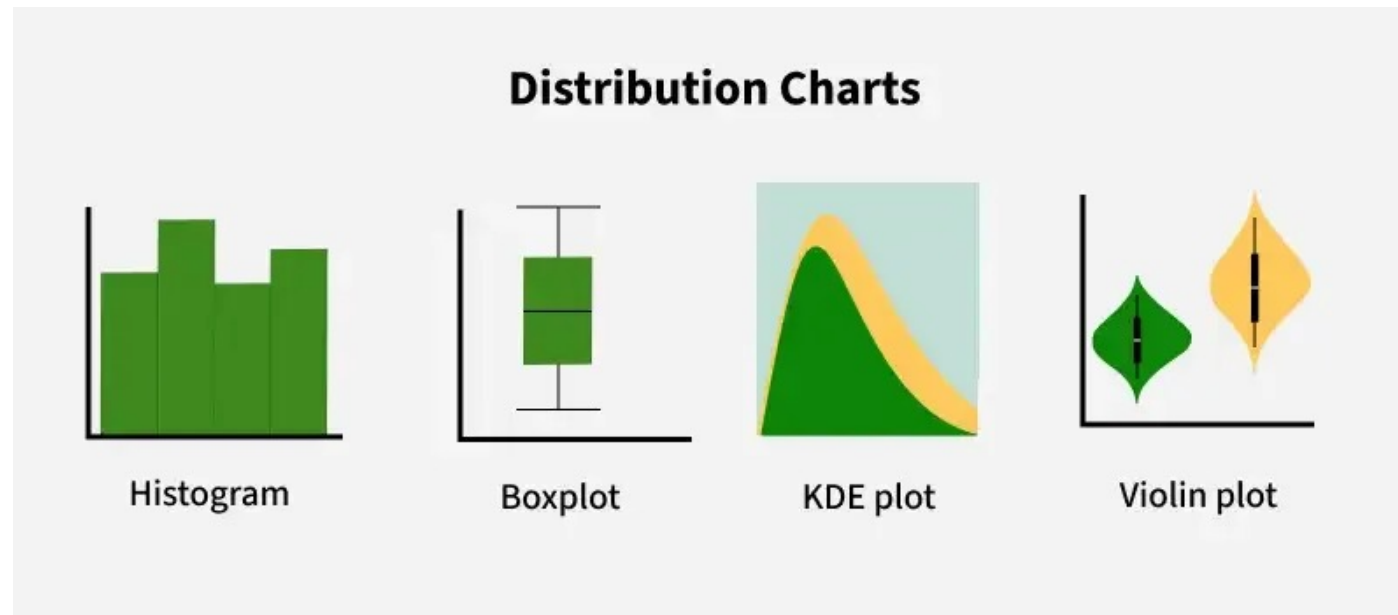
- **Scatter Chart:** Plots two variables to show their relationship or trend.
- **Bubble Chart:** Adds size and color to scatter plots for more variable comparison.
- **Heatmap:** Uses color in grids to reveal patterns, density or relationships in data.



Distribution Charts

Distribution charts show how data is spread across ranges or categories. They help identify patterns, outliers and overall trends

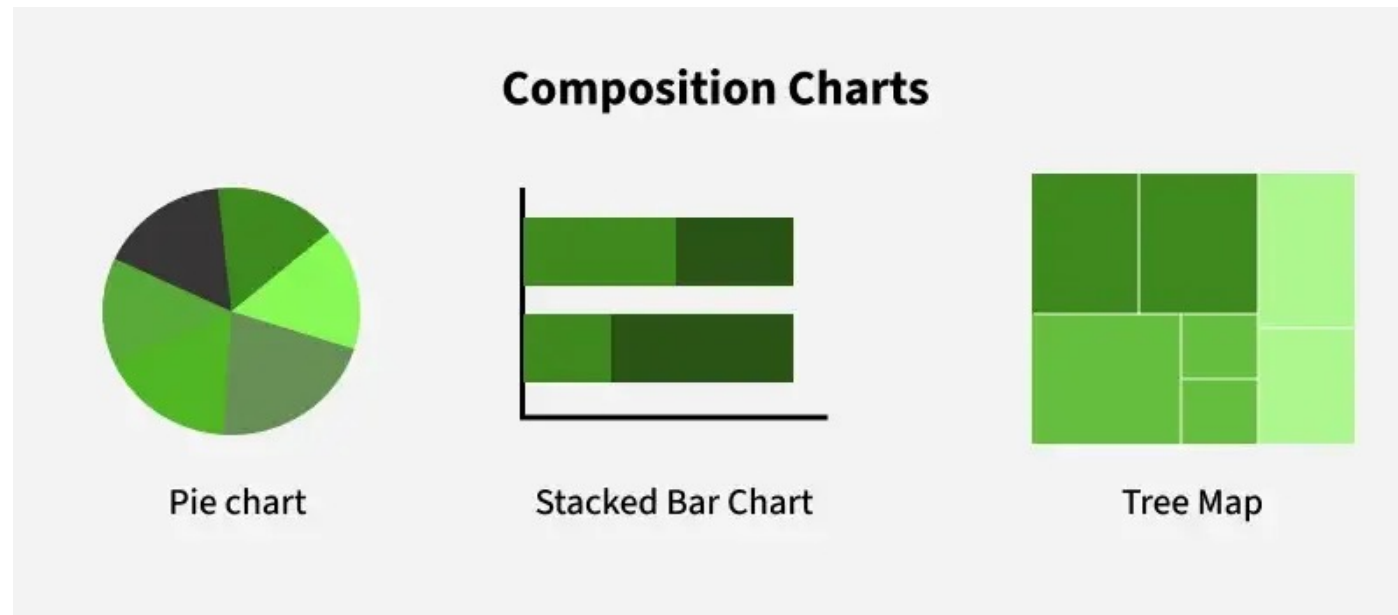
- **Histogram:** Shows frequency of values within ranges.
- **Box Plot:** Displays data spread, median and outliers.
- **KDE Plot:** Smooth curve showing data distribution shape.
- **Violin Plot:** Combines box plot and density curve to compare group distributions



Composition Charts

Composition charts break down a whole into its individual components, showing how each part contributes to the total. They also help track how these contributions change over time offering insights into structure and shifts.

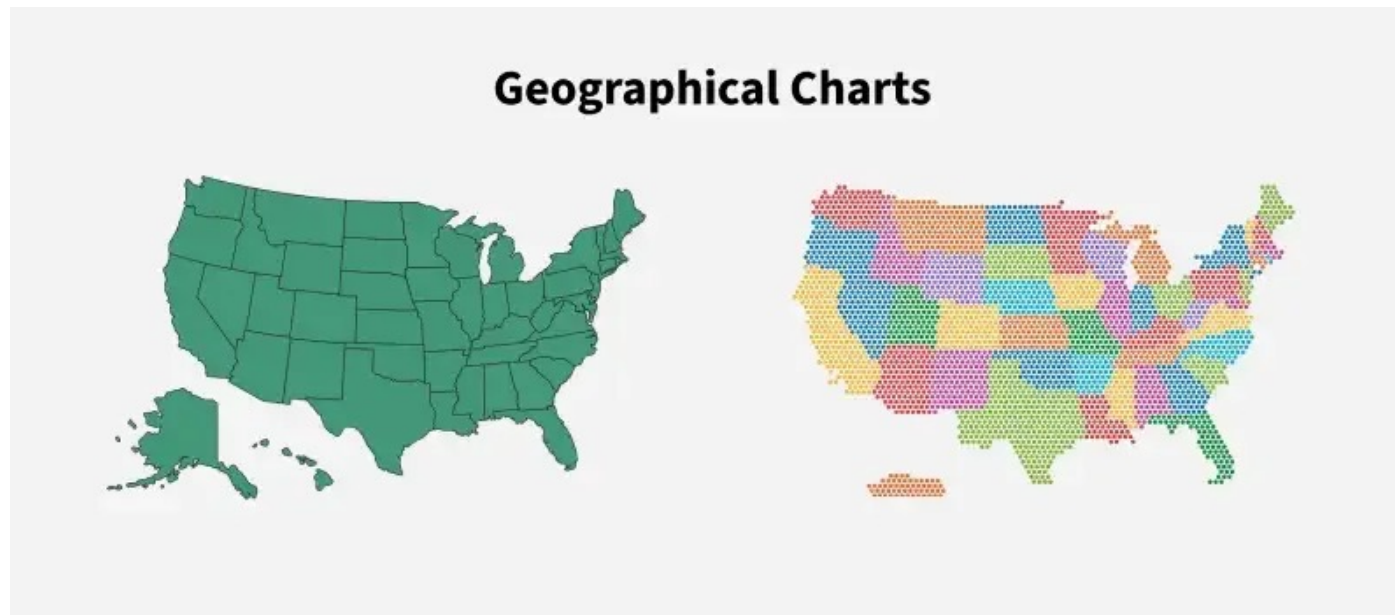
- **Pie Chart:** Best for small datasets shows each part's share of the total.
- **Stacked Bar Chart:** Compares parts across categories within bars.
- **Treemap:** Uses nested boxes sized by value to show part-to-whole relationships.



Geographical Charts

Geographical charts are used to visualize data linked to specific regions or locations. They help identify spatial patterns, regional differences and trends across areas using maps and color coding.

- Choropleth Map: Uses color to show values (like population or income) across regions.
- Cartogram: Resizes regions based on data values (e.g., population) distorting geography for emphasised.



Other Rules

- **Audience-Centric Design:** Tailor visualizations to our audience's knowledge. A technical audience may need detailed graphs while a general audience benefits from simpler charts.
- **Design Clarity and Consistency:** Choose the right chart for our data and keep the design clean with consistent colors, fonts and labels and also avoid clutter to ensure clarity.
- **Provide Context:** Always provide context by including labels, titles and data source acknowledgments. This helps viewers to understand the significance of the data and builds trust in the results.

Tutorial

